UiA University of Agder

Bachelor

Developing a Storage Solution for Research Data

Supporting FAIR Principles Using a Ceph-Based Storage Backend

ABDIQADIR ISMAIL ABDI

SUPERVISOR
Sigurd Brinch

University of Agder, 2026
Faculty of Engineering and Science
Department of Engineering and Sciences

Gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk.

1. Vi erklærer herved at denne besvarelsen er vårt eget arbeid, og at vi ikke har brukt andre kilder eller mottatt annen hjelp enn det som er nevnt i besvarelsen.
2. Denne besvarelsen:
    - har ikke vært brukt til annen eksamen,
    - refererer ikke til andres arbeid uten korrekt kildehenvisning,
    - refererer ikke til eget tidligere arbeid uten at dette er oppgitt,
    - inneholder alle referanser i litteraturlisten,
    - er ikke en kopi av andres arbeid.
3. Vi er kjent med at brudd på ovennevnte regnes som fusk og kan medføre annullering av eksamen og utestengelse.
4. Vi er kjent med at oppgaven kan bli plagiatkontrollert.

Student: **Abdiqadir Ismail Abdi**
Group: **BPR/D-026-35 (BPR/D-026-)**
Year: **2026**

Acknowledgements

Abstract

Contents

List of Figures

List of Tables

Abbreviations and Glossary

Chapter 1

Introduction

1.1 Background

1.2 Problem Statement

1.3 Aim and Objectives

1.4 Scope and Delimitations

1.5 Research Questions

1.6 Method Overview

1.7 Report Structure

Chapter 2

Context and Needs

2.1 Problem Context and Stakeholders

The university, as a center of research, naturally produces a considerable amount of data that must be stored either temporarily or permanently. This data may take different forms, such as raw measurements or processed datasets. A primary concern for the university is ensuring that this continuous flow of research data is stored safely in terms of capacity, cost efficiency, and long-term durability.

While the optimization of storage infrastructure itself has largely been successful, the purpose of storing research data extends beyond mere archival. Research data is intended to be reused, shared, and built upon. This raises the question of how effectively such data can be retrieved and reused over time. When data is stored without being properly registered, described, or organized, identifying what data exists, who created it, and how it can be accessed becomes increasingly difficult. Under these conditions, retrieving relevant datasets can become a significant challenge for researchers.

The lack of structure introduces additional issues. Without descriptive metadata, datasets lose their context and meaning. Furthermore, without clear access control mechanisms, it becomes unclear who is authorized to read or modify data, increasing the risk of accidental or unauthorized changes. In addition, the absence of versioning means that modifications may overwrite previous states, making it difficult or impossible to trace changes or reproduce earlier results.

These challenges highlight the need for an approach in which research data is registered upon arrival, enriched with metadata describing its content and origin, and associated with clearly defined access rights. Keeping track of different versions of datasets is also essential in order to avoid data loss and confusion when changes occur. Regarding access control, permissions should be assigned based on roles rather than individually. For example, researchers may require both read and write access to their data, students may be granted read-only access, dataset owners should have full control, and external

collaborators should have limited access depending on their role.

2.2 Research Data Workflows

To understand the expectations of a FAIR-supported storage system, it is important to grasp how research data is actually handled in practice. This section will describe a universities average workflow through the eyes of a researcher, based on solutions used for handling sensitive research data, such as TSD (Service for Sensitive Data).

2.2.1 Data creation

Research data is often a result of empirical studies, such as questionnairs, interviews and analysis of existing data. Generally, the data is obtained by researchers or research groups over a period of time.

The metadata, which is the information on how data is collected, which versions exist and which variables are relevant, are noted to various degrees during this stage. Without a proper storage plan however, the metadata is easily lost during the project, which effects sharing and reuse of data down the line.

2.2.2 Storage

Once a project has some data, it is saved in a storage accepted by the university, for example TSD which is widely accepted in Norwegian universities. Such storages give researches a safe and secure storage system that meet the requirements for privacy and information security.

For researchers, the storage system seems safe but less flexible than preferred. Data is usually organised in file systems that expand and develop over time, without the proper guidelines for structure. Versioning of files is often a manual task, for example by creating copies with new filenames. This creates a rather frustrating issue, making it hard to know which version is the newest or which was used for analysis and publication.

2.2.3 Sharing

Sharing data is usually an internal affair within the research group, with external partners. In TSD it is a requirement that access is explicitly given, which can be time-consuming for researchers without the technical background required.

It can be difficult to understand who has access to what data, and how long the access granted will last. Sharing is also often connected to the active phase of the project, meaning when a project is concluded there is a lack of focus on accessibility, and data can be inaccessible for others.

Metadata frequently lacks a structured way of being shared, which makes it harder for newer users to understand the content without direct communication with the owner(s) of the data.

2.2.4 Reuse

There are numerous issues that arise when attempting to reuse data from previous projects. Even when data is securely stored, missing or incomplete documentation can create problems with understanding how the data can be used once more.

Reuse of data can be challenging if certain aspects of data is unclear, such as which version an analysis belongs to, or which limitations exist for further use, which regularly leads to data being looked over. Another issue with unclear documentation is new researchers needing to spend more time reconstructing the context and structure of data before being able to use it.

2.3 Constraints and Assumptions

2.3.1 Constraints

- **Institutional Policies**
  The system must deal with the universities internal guidelines for handling research data, including the requirements for accepted storage solutions and data security.

- **Jurisdictional and Regulatory Requirements**
  Handling of research data must follow international laws and regulations, such as privacy regulations and ethical requirements. These limitations limits how data can be stored, shared and be available.

- **Handling of Sensitive Data**
  The system must be able to be used for storage of sensitive research data, which requires strict guidelines for confidentiality, access and traceability.

- **Organisational Practices**
  Routines for handling research data varies between research groups and projects. Therefore, the system cannot assume uniform practices or standardized data structures across users.

- **Project Scope and Time Limitations**
  The bachelor project is limited by available time and resources, which sets limitations on complexity and functionality.

2.3.2 Assumptions

- **User Competence**
  It is expected that the systems primary users are researches with limited technical competence. The system can therefore not be based on users with greater understanding of technology.
- **Variation Within Data Types**
  Research will have varied formats, size, and structure, determined by the area of study and their method.
- **Metadata Quality**
  It is expected that metadata will be created manually by researchers, and this information can be incomplete or inconsistent.
- **Collaboration Patterns**
  It is expected that the sharing of research data will mainly be executed within research groups or with a limited amount of research partners.
- **Existing Infrastructure**
  It is assumed that the system is part of an existing IT infrastructure and must collaborate with other approved tools and services.

2.4 Non-functional Expectations

2.4.1 Traceability

It is expected that the system provides an overview of how research data is handled over time. This includes maintaining relationships between datasets, metadata, and changes made throughout the project lifecycle. Traceability is important to understand how data has been utilised in analysis and publications to provide options for re-use.

2.4.2 Data Integrity

There is an expectation that the system supports research staying consistent unless changes are intended. Researchers are expected to have confidence in the system's ability to save and maintain data without the threat of changed or damaged data. Data integrity is very important for preservation of data quality and re-use.

2.4.3 Auditability

It is expected that the handling of research data can be reviewed and assessed at a later date. This includes knowing who has had access to the data, and how the data

has been used or modified over time. Auditability is relevant for both internal control, collaboration and maintaining institutional requirements.

2.4.4 Governance

The system is expected to support a clear framework for responsibilities and ownership of research data. This includes how access is shared and revoked, how data is managed after a project is closed, and how institutional guidelines can survive time. A good structure leads to reasonable expectations and sustainable use of data.

2.4.5 Usability

Despite addressing complex requirements, the system is expected to address complex requirements, whilst maintaining an understandable and applicable system for researchers with little technical background. Trust in the system is important for researchers, where the system follows recommended practices for storage, sharing, and documentation of data.

Chapter 3

FAIR Principles and Research Data Management

3.1 Overview of FAIR Principles

The FAIR principles, which stand for Findable, Accessible, Interoperable and Reusable, are a set of guidelines for data management. These principles are intended to ensure that data can be used effectively over time across different systems, sectors and subject areas. The principles were developed in response to the increasing challenge within the data and research community in which large amounts of data exist but are hard to find, understand, combine, or reuse.

The core purpose of FAIR principles is that data should no longer just be stored, but rather facilitated for use. These principles do not concern themselves for specific tools or platforms, but rather provide a well designed architecture for structure, documentation and standardising. As a result, FAIR can be used across multiple subject areas, such as science, medicine and social science.

It is important to note that using FAIR doesn't require data to be publicly available. However, information about the data should be, meaning that the public should be able to access a description of the data, who made it, when it was collected, and how the data can be used. Certain data can require the correct permissions to gain access, which require set conditions such as who can apply for permission, what criteria is necessary to gain access, and relevant jurisdictional constraints.

When applied to systems, the FAIR principles provide a useful perspective for interpretation rather than a checklist for compliance. They help creating how systems support discovery, access, exchange, and reuse of data across organisational boundaries. This perspective allows FAIR to act as a conceptual bridge between data management theory and practical system design and implementation which we will delve into in later sections.

3.2 FAIR Principles as System Requirements

3.2.1 Findable

Theoretical Perspective

Findability requires that research data and metadata are assigned persistent identifiers that make them discoverable through search mechanisms over time.

Implications for the System

For the proposed system, findability means that every dataset must be registered as a distinct entity, associated with descriptive metadata, and identifiable through a unique identifier.

Consequences of Missing Findability

Without findability, datasets may exist but remain undiscovered, leading to duplicated work or loss of valuable data.

3.2.2 Accessible

Theoretical Perspective

Accessibility refers to retrieving data through standardized and secure mechanisms with clearly defined access conditions.

Implications for the System

Access must be controlled through authentication and authorization, while metadata remains accessible even when data is restricted.

Consequences of Missing Accessibility

Improper accessibility can lead to security risks or hinder legitimate research activities.

3.2.3 Interoperable

Theoretical Perspective

Interoperability enables data integration across systems using shared formats and vocabularies.

Implications for the System

Metadata must rely on standardized schemas and avoid project-specific conventions.

Consequences of Missing Interoperability

Data becomes isolated and difficult to reuse across disciplines.

3.2.4 Reusable

Theoretical Perspective

Reusability requires rich metadata, provenance, and clear usage conditions.

Implications for the System

The system must preserve version history and descriptive context over time.

Consequences of Missing Reusability

Poorly documented data leads to wasted resources and reduced scientific value.

3.3 FAIR Compliance Goals for the Project

The aim for this Bachelor's project is to create a storage solution for research data which supports the FAIR principles. To achieve this goal, the assignment requires good handling, sharing and reusing through verifiable properties.

3.3.1 Findable

The Goal

The system shall support that saved research data is unambiguous and searchable.

1. Each data set shall be linked to a unique identifier
2. It must be possible to associate metadata with each data set
3. Metadata must be structured so that they can be identified and used for searching

3.3.2 Accessible

The Goal

The system must make research available in a controlled and expected way.

1. There should be clear defined mechanisms for accessing data
2. Access to data should be limited based on user roles and rights
3. Metadata should stay available even if the data is not accessible

3.3.3 Interoperable

The Goal

The system should support the usage of standerdised formats and descriptions that enable interaction between other systems.

1. Metadata must be stored in a structured and machine-readable format
2. The storage solution cannot lock data to proprietary formats
3. It should be possible to integrate the system with external tools and services

3.3.4 Reusable

The Goal

The system should allow for the reuse of research data over any given time.

1. Metadata should contain enough information to understand the datas content and context
2. The system should allow the attachment of licenses or terms of use to data
3. The storage system should support storing data over time without losing integrity

Chapter 4

Existing Solutions and Related Work

4.1 Categories of Research Data Management Solutions

4.2 Evaluation Criteria

4.3 Comparative Analysis

4.4 Gap Analysis

4.5 Justification of the Selected Approach

Chapter 5

Theoretical and Technical Foundations

5.1 Storage Interface Models

5.2 Ceph as a Storage Backend

5.3 Core Concepts

5.3.1 Data and Metadata

5.3.2 Access Control

5.3.3 Auditing and Provenance

5.3.4 Versioning and Immutability

5.3.5 Annotation of Research Data

Chapter 6

Requirements Specification

6.1 Requirements Elicitation Method

During the elicitation of the requirements for the proposed system, rather than basing the work on a predefined technical solution, requirements were derived from an analysis of the problem context, stakeholder needs, and the FAIR principles as applied to research data management.

First, key challenges faced by researchers when storing and managing research data were identified based on the problem context described in Chapter 2. These challenges include difficulties in data discovery, lack of descriptive metadata, unclear access rights, absence of versioning mechanisms, and limited traceability of data usage. Stakeholder roles such as researchers, collaborators, students, and external users were analyzed in order to clarify responsibilities and expectations related to data access and modification.

Second, the FAIR principles introduced in Chapter 3 were treated as compliance obligations that the system must satisfy. Each principle (Findable, Accessible, Interoperable, and Reusable) was translated from its theoretical definition into concrete system behaviors and applied systematically during the requirements derivation process.

Based on this analysis, requirements were formulated by identifying what the system must do in order to address the observed problems and satisfy FAIR-related obligations. Functional requirements describe the capabilities the system must provide, such as dataset registration, metadata management, access control, versioning, and auditing. Non-functional requirements capture quality attributes and governance concerns, including data integrity, traceability, immutability of published data, and auditability of access decisions.

All requirements are expressed in a clear and testable form using "The system shall ..." statements. This ensures that requirements can be traced forward to design and implementation decisions and later verified during system evaluation. The elicitation process thus establishes a clear link between the identified problems, FAIR principles,

and the resulting system requirements.

6.2 Functional Requirements

FR1 — Dataset Registration
The system shall allow users to register a dataset as a distinct entity before or during data storage.

FR2 — Dataset Identification
The system shall assign a unique identifier to each registered dataset.

FR3 — Metadata Management
The system shall allow users to create, view, and update descriptive metadata associated with a dataset.

FR4 — Mandatory Core Metadata
The system shall enforce the presence of a minimum set of mandatory metadata fields for each dataset before it can be registered.

FR5 — Dataset Discovery
The system shall allow users to search for datasets using metadata attributes such as title, creator, keywords, or associated project.

FR6 — Role-Based Access Control
The system shall enforce access control based on predefined user roles.

FR7 — Access Rights Definition
The system shall allow dataset owners to define read and write permissions for other users or roles.

FR8 — Controlled Data Access
The system shall ensure that only authorized users can access or modify a dataset according to assigned permissions.

FR9 — Dataset Versioning
The system shall create a new version of a dataset whenever its content is modified.

FR10 — Version Preservation
The system shall preserve previous versions of a dataset and prevent them from being overwritten.

FR11 — Version Retrieval
The system shall allow users to access and retrieve previous versions of a dataset.

FR12 — Access and Modification Logging
The system shall log all dataset access and modification events.

FR13 — Audit Log Retrieval
The system shall allow authorized users to view audit logs related to dataset access and modification activities.

FR14 — Dataset Annotation
The system shall allow users to attach annotations to datasets without modifying the dataset content.
FR15 — Metadata Accessibility
The system shall ensure that dataset metadata remains accessible even when access to the dataset content is restricted.

6.3 Non-functional Requirements

NFR1 — Data Integrity
The system shall ensure that stored dataset content is not altered unintentionally during metadata operations or access control changes.
NFR2 — Immutability of Published Versions
The system shall ensure that published dataset versions remain immutable and cannot be modified once released.
NFR3 — Traceability
The system shall ensure that all dataset changes, including metadata updates and version creation, are traceable to a specific user and timestamp.
NFR4 — Auditability
The system shall ensure that all access and modification events are auditable and can be reviewed by authorized users.
NFR5 — Separation of Data and Metadata
The system shall manage metadata independently of the underlying dataset content to avoid unintended data modification.
NFR6 — Access Control Enforcement
The system shall consistently enforce access control decisions across all system interfaces.
NFR7 — FAIR Compliance
The system shall support the FAIR principles by ensuring that datasets remain findable, accessible under defined conditions, interoperable, and reusable throughout their lifecycle.
NFR8 — Scalability Independence
The system shall scale independently of the underlying storage capacity provided by the Ceph backend.
NFR9 — Reliability of Metadata Services
The system shall ensure that metadata and access control services remain available even when dataset content access is restricted.
NFR10 — Governance and Policy Compliance
The system shall support institutional governance policies by enabling controlled access, auditing, and long-term preservation of research data.

6.4 Acceptance Criteria

6.5 Requirements Traceability

Chapter 7

System Design

7.1 Design Goals and Principles

The system was designed based on the Chapter 6 discussion related to many known issues concerning the management of research data storing at the University of Agder. Therefore, the main purpose of the design choice was to create a neat and organized way to handle that considerable amount of data in an efficient way. One way of addressing this issue was to build a new layer on top of the current Ceph storage while avoiding interference with the storage's own activities.

Consequently, designing and building the system should follow the FAIR principles requirements, such as ensuring that researchers can actually find data using simple descriptions. Furthermore, access to a given dataset should, on the one hand, be straightforward for the right person or group of persons through role-based access control. Finally, datasets have to be stored in an efficient way that allows future reuse. These requirements were not just ideas or thoughts, but real commitments that guided every choice made in the design of the system.

As mentioned earlier, Ceph activities are kept totally separated from the smart storage system, which operates efficiently at the entry point of the storage. One crucial task is to keep track of files and directories and maintain a log of who consults them. Another no less important task is to preserve earlier versions of data whenever modifications occur, in order to avoid accidental overwriting or to allow access to original versions if needed. For that reason, users have access to historical metadata related to the datasets.

Modularity and expandability were also guiding principles in the overall design of the system. For this reason, each technical unit was kept as a separate module, in such a way that updates or expansions would only affect the relevant unit.

7.2 Architecture Overview

In this section will be introduced a layered architecture in which responsibilities related data storage, metadata management, access control, and governance are clearly separated. This architectural methodology is preferred in order to preserve the stability and scalability of the current CEPH storage infrastructure while introducing a higher-level research data management capability required for FAIR compliance.

The CEPH is used as a backend for storing efficiently raw datasets content and therefore lie at the bottom of the architecture. That design choice implies that CEPH is not aware nor care of concepts such metadata, access rights, versions or even users in the research sense. Such good and efficient CEPH's storage job should be kept undisturbed by governance logic, so that each subpart of the system accomplishes its duty separately.

Above the storage layer lies the smart storage system, which forms the core of the proposed solution. This application layer behaves as an intermediary between users and the CEPH backend and furthermore is responsible for enforcing policies related to all research data management. Its main tasks include dataset registration, metadata management, role-based access control, data versioning, and auditing of access and modification events. All contacts with CEPH goes through this layer, guaranteeing that storage operations are always governed by the defined rules and constraints.

The metadata and governance information run by the application layer is persisted in a dedicated PostgreSQL database. This database houses descriptive metadata, dataset identifiers, access control rules, version relationships, and audit logs. PostgreSQL does not have direct interaction with the CEPH storage but get accessed only through the application layer. This strict separation certifies that metadata operations cannot accidentally affect stored dataset content and thus that governance logic remains centralized.

System seen from a user perspective, all interaction is performed by the use of defined interfaces exposed by the application layer. At the registration of the new datasets, the system creates a logical entity, assigns a unique identifier, and associates it with metadata and access rules stored in PostgreSQL. When dataset content is either uploaded or modified, the application layer controls permissions, stores the data in CEPH and then records the corresponding version and audit information. Reading access follows a similar path as well, certifying that authorization checks and logging are consistently enforced.

This architectural design establishes clear boundaries between storage, governance, and metadata management. It ensures that FAIR-related obligations such as findability, controlled accessibility, traceability, and reusability are enforced independently of the underlying storage technology. At the same time, the modular structure allows future extensions or replacements of individual components without requiring fundamental changes to the overall system architecture.

7.3 Component Design

Build on the layer organized architecture described in the last section, the smart storage system is designed in a set of logical components where each one responsible for a specific feature of research data management. This component-build design follows the principles of modularity and separation of concerns which guaranty that individual responsibilities are well-defined and that modifications to one component have minimal impact on others.

The interface element runs the entry point where users interact with the system. It displays the available operations related to data registration, access, modifications, and forward requests to the suitable internal modules.

A dedicated component handles metadata management by storing, retrieving and updating descriptive information related to datasets. Thus, this component makes sure that metadata is managed regardless of datasets content and in the other hands support its discoverabilities and long-term interpretabilities.

Access control is managed by a specific authorization component that enforces role-based access control policies. The component analyzes user roles and permissions assigned before any operation is performed on a dataset.

A versioning component manages data versioning by creating and keeping distinct dataset versions whenever changes happens. This feature safeguards thus, that previous versions are retained and beyond that, supports traceability and reproducibility.

A logging component provide auditing and traceability by recording datasets access and modifications events together with user identities and timestamps.

Finally, interaction with the Ceph storage backend is provided through the storage adapter component. This component is responsible for data transfer operations after authorization and versioning decisions have been taken, thus ensuring that Ceph remains a passive storage backend without any governance logic.

These components provide a clear and uncluttered structure for the application layer that can be used to provide FAIR-compliant research data management.

7.4 Data Model Design

7.4.1 Conceptual Data Model

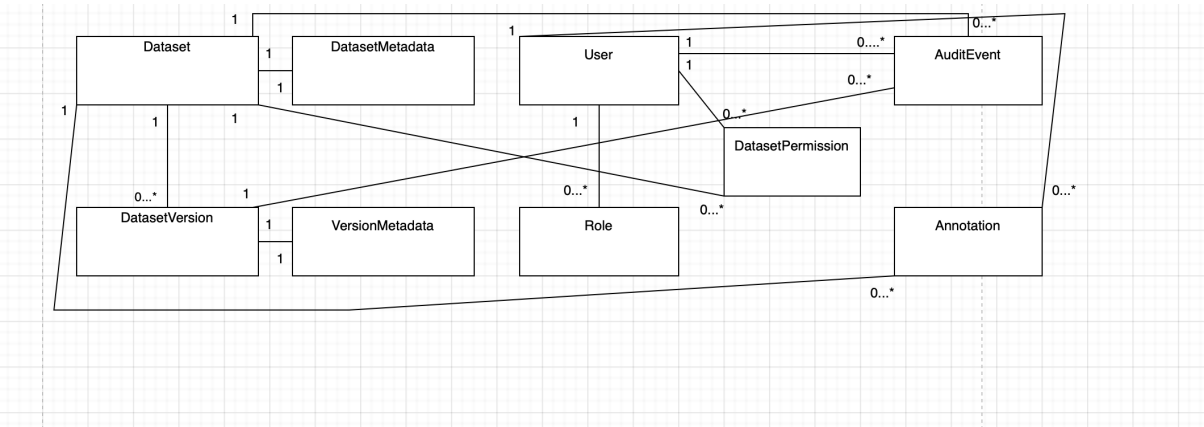

Figure 7.1: Conceptual data model for the Research Data Management (RDM) system.

Figure 7.1 presents the conceptual data model of the projected RDM system. It states the main entities necessary to satisfy the system requirements such as Dataset and DataVersion which represents raw data and their different versions. Separation of data and metadata is enhanced through use of separate metadata entities. The model also incorporates governance related entities like User, Role, DatasetPermissions, AuditEvent, and Annotation, which allow role-based access control, auditing, and non-destructive annotations on datasets.

7.4.2 System Architecture

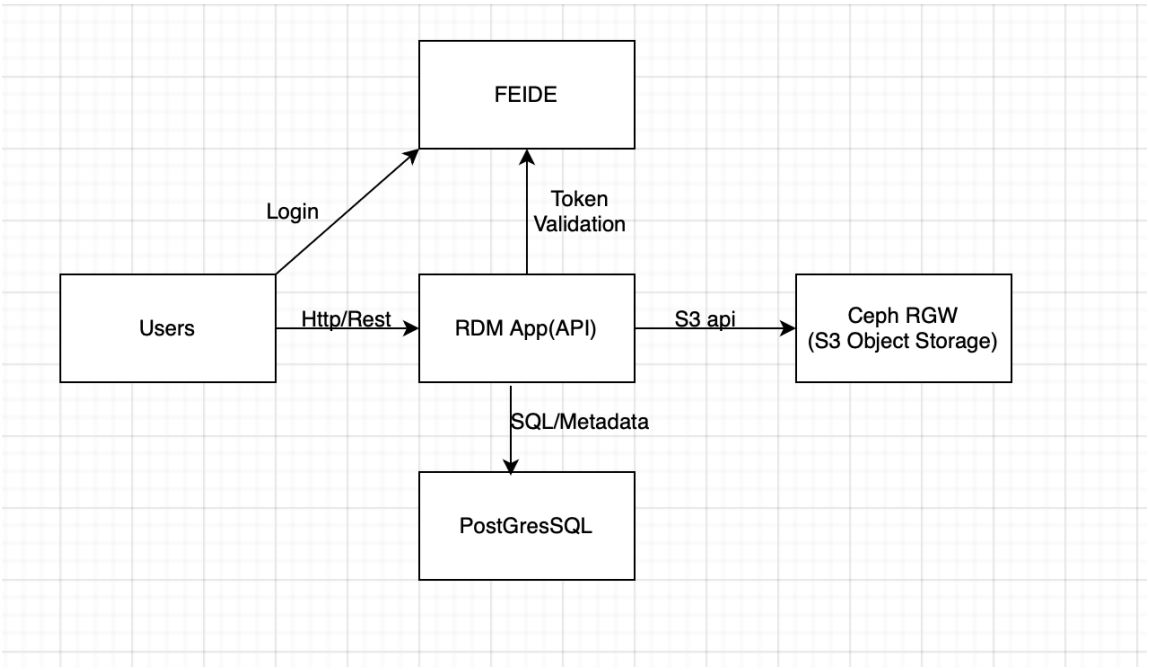

Figure 7.2: High-level system architecture of the proposed solution.

Figure 7.2 shows the high-level architecture of the solution, thus Users interact with the RDM application by an HTTP/REST API, while authentication is handled through FEIDE and later validated by the application. The application takes care of governance information such as dataset records, metadata, permissions, audit events, and annotations, inside a PostgreSQL database. Consequently, the actual dataset content is stored in Ceph via the S3-compatible interface offered by Ceph RGW. Such architecture keeps the raw storage responsibilities in Ceph but the application layer handles FAIR-oriented data management, access control, versioning logic, and auditing.

7.4.3 Dataset Lifecycle Workflow

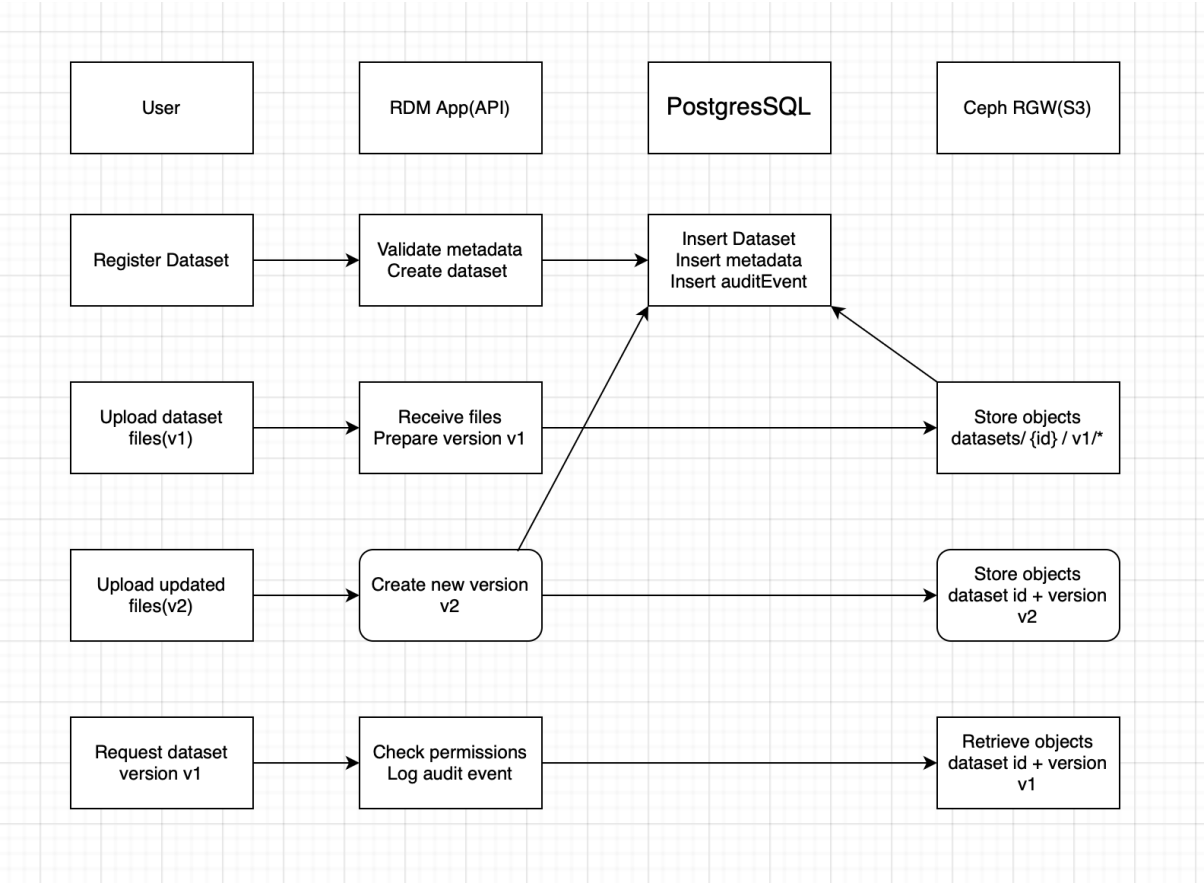

Figure 7.3: Dataset lifecycle workflow showing registration, versioning, storage, and retrieval.

Figure 7.3 shows the central workflow supported by the system. First, a user registers a dataset by adding the required metadata. Then the application validates the metadata and stores the dataset record in the database, joining and audit event. Next, the user uploads the initial dataset content that is already stored in Ceph as version1, concurrently the version information and audit logs are stored in the database. Once or if dataset changes, the user uploads updated content and the application creates a new version, stores the new objects in Ceph and then records the version in the database. Lastly, when a user requests a given version, the application controls the permissions, logs the access and retrieves the requested object from Ceph, guaranteeing traceability and controlled access throughout the dataset lifecycle.

7.5 Storage Mapping Strategy

7.6 Interface Design

7.7 Security and Privacy Design

7.8 Design Decisions and Tradeoffs

Chapter 8

Implementation Overview

8.1 Technology Choices

8.2 System Modules

8.3 Key Workflows

8.4 Data Integrity and Consistency

8.5 Failure Handling

Chapter 9

Evaluation

9.1 Evaluation Strategy

9.2 Functional Validation

9.3 FAIR Compliance Evaluation

9.4 Security Evaluation

9.5 Performance Observations

9.6 Usability Considerations

Chapter 10

Discussion

10.1 Strengths of the Proposed Solution

10.2 Limitations and Tradeoffs

10.3 Interface and Integration Discussion

10.4 Governance and Adoption Considerations

10.5 Lessons Learned

Chapter 11

Future Work

11.1 Functional Extensions

11.2 Scalability and Automation

11.3 Governance and Policy Enhancements

Chapter 12

Conclusion

12.1 Summary of Contributions

12.2 Answers to Research Questions

12.3 Final Remarks

Bibliography

Appendix A

Glossary

Appendix B

Requirements Traceability Matrix

Appendix C

API Specification

Appendix D

Data Model Details

Appendix E

Test Cases and Results

Appendix F

Risk Register